Universidade Federal de Sergipe — Departamento de Computação
COMP0498 — Algoritmos e Estruturas de Dados II · Turma T02 · 2026.2

Lista de Exercícios — Divisão e conquista e programação dinâmica

Prof. Dr. André Yoshiaki Kashiwabara · andreyoshiaki@dcomp.ufs.br

Convenções. $\lg n$ é $\log_2 n$. Nas recorrências, n é uma potência da base sempre que for preciso, e $T(1) = \Theta(1)$. Nos problemas de programação dinâmica, siga a notação das aulas: $\mathrm{OPT}(j)$ e $p(j)$ nos intervalos ponderados; $\mathrm{OPT}(i, c)$ na mochila; $c[i, j]$ na maior subsequência comum e $E[i, j]$ na distância de edição. Nos grafos, os vértices são $0, 1, \ldots, n-1$, e $R^{(k)}$ e $D^{(k)}$ só admitem vértices intermediários em $\{0, \ldots, k-1\}$; assim, $R^{(0)}$ e $D^{(0)}$ são a matriz de adjacências e a matriz de pesos, e $R^{(n)}$ e $D^{(n)}$ são a resposta. As instâncias desta lista são *diferentes* das usadas em aula: refaça as contas em vez de reconhecer o resultado.

Parte I — Divisão e conquista

Exercício 1. Teorema mestre: aplicar, desenhar e saber quando não serve.

Use a versão do teorema mestre vista em aula: se $T(n) = a\,T(n/b) + f(n)$ com $a \geq 1$, $b \geq 2$ e $f(n) \in \Theta(n^d)$, $d \geq 0$, então $T(n)$ é $\Theta(n^d)$, $\Theta(n^d \log n)$ ou $\Theta(n^{\log_b a})$ conforme $a < b^d$, $a = b^d$ ou $a > b^d$.

(a) Resolva cada recorrência, indicando a, b, d e o caso aplicado: *(i)* $T(n) = 3\,T(n/2) + \Theta(n)$; *(ii)* $T(n) = 7\,T(n/2) + \Theta(n^2)$; *(iii)* $T(n) = 8\,T(n/2) + \Theta(n^2)$; *(iv)* $T(n) = 4\,T(n/2) + \Theta(n^2)$; *(v)* $T(n) = 2\,T(n/4) + \Theta(n)$; *(vi)* $T(n) = 2\,T(n/2) + \Theta(1)$. Diga a qual algoritmo da unidade correspondem as três primeiras.

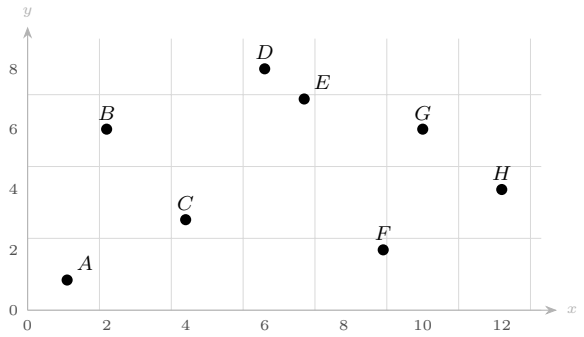
(b) Desenhe os três primeiros níveis da árvore de recursão de $T(n) = 3\,T(n/2) + n$, escreva o custo do nível j e some os níveis para chegar ao mesmo resultado do item (a) (i). Qual parte da árvore domina o custo: a raiz, todos os níveis por igual, ou as folhas?

(c) Para cada recorrência abaixo, explique por que a versão acima do teorema mestre *não* se aplica e resolva-a por outro caminho: *(i)* $T(n) = T(n-1) + \Theta(n)$; *(ii)* $T(n) = 2\,T(n/2) + \Theta(n \lg n)$; *(iii)* $T(n) = T(n/3) + T(2n/3) + \Theta(n)$.

(d) Um algoritmo divide uma entrada de tamanho n em a subproblemas de tamanho $n/4$ e combina as respostas em $\Theta(n)$. Para quais valores inteiros de a ele é assintoticamente mais rápido que um algoritmo $\Theta(n^2)$? Dê o custo em função de a em cada faixa.

Exercício 2. Par de pontos mais próximo.

Considere os oito pontos abaixo: $A(1,1)$, $B(2,6)$, $C(4,3)$, $D(6,8)$, $E(7,7)$, $F(9,2)$, $G(10,6)$, $H(12,4)$.



(a) Quantas distâncias a força bruta calcula para n pontos? E para estes oito?

(b) *Aquecimento na reta.* Projete os pontos no eixo x. Descreva um algoritmo $\Theta(n \log n)$ para o par mais próximo em uma reta e explique por que, nela, a combinação da divisão e conquista só precisa olhar *um* par que atravessa a linha divisória.

(c) Execute o nível mais alto do algoritmo de divisão e conquista no plano: divida os pontos pela mediana em x, usando como linha divisória a reta $x = 6{,}5$; diga qual é o par mais próximo de cada metade e o valor de $\delta = \min(\delta_E, \delta_D)$, liste os pontos que caem na faixa de largura 2δ em torno da linha divisória, ordenados por y, e mostre quais comparações a faixa faz e qual é o par devolvido.

(d) Explique por que, na faixa ordenada por y, cada ponto só precisa ser comparado com um número *constante* de sucessores. Que figura geométrica sustenta esse argumento?

(e) Escreva a recorrência do algoritmo em duas versões: *(i)* cada chamada ordena a sua faixa por y; *(ii)* os pontos são ordenados por y uma única vez, antes da primeira chamada, e a ordem é mantida nas chamadas. Resolva as duas.

Exercício 3. Multiplicação de inteiros grandes (Karatsuba).

Lembre o algoritmo: com $x = a \cdot 10^m + b$ e $y = c \cdot 10^m + d$, calcule $z_2 = ac$, $z_0 = bd$, $z_1 = (a+b)(c+d) - z_2 - z_0$ e devolva $xy = z_2 \cdot 10^{2m} + z_1 \cdot 10^m + z_0$.

(a) Calcule 1234×5678 pelo algoritmo de Karatsuba com $m = 2$, mostrando a, b, c, d, z_2, z_0, z_1 e a montagem final. Um dos fatores de z_1 tem mais dígitos que os outros: qual, e por que isso não estraga a análise?

(b) A divisão ingênua calcula os quatro produtos ac, ad, bc e bd. Escreva a recorrência das duas versões e resolva-as pelo teorema mestre. Por que a ingênua não ganha nada do algoritmo escolar?

(c) Levando a recursão até 1 dígito, quantas multiplicações de um dígito cada versão faz para $n = 2^k$ dígitos? Compare os dois números para $n = 1024$.

(d) A mesma ideia vale para números complexos: o produto $(p + q\,i)(r + s\,i) = (pr - qs) + (ps + qr)\,i$ usa quatro multiplicações reais. Mostre como obtê-lo com *três*.

(e) Na prática, as bibliotecas de inteiros grandes não levam a recursão de Karatsuba até 1 dígito. Por quê? Como se escolhe o ponto de parada?

Exercício 4. Multiplicação de matrizes (Strassen).

Os sete produtos de Strassen, para $C = AB$ com blocos A_{ij} e B_{ij}, são

$$\begin{aligned} &M_1 = (A_{11} + A_{22})(B_{11} + B_{22}), \quad M_2 = (A_{21} + A_{22})\,B_{11}, \quad M_3 = A_{11}\,(B_{12} - B_{22}),\\ &M_4 = A_{22}\,(B_{21} - B_{11}), \qquad\quad M_5 = (A_{11} + A_{12})\,B_{22}, \quad M_6 = (A_{21} - A_{11})(B_{11} + B_{12}),\\ &M_7 = (A_{12} - A_{22})(B_{21} + B_{22}), \end{aligned}$$

com $C_{11} = M_1 + M_4 - M_5 + M_7$, $C_{12} = M_3 + M_5$, $C_{21} = M_2 + M_4$ e $C_{22} = M_1 - M_2 + M_3 + M_6$.

(a) Calcule AB para $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ e $B = \begin{pmatrix} 1 & 4 \\ 2 & 5 \end{pmatrix}$ pelos sete produtos e confira com o produto direto.

(b) Prove, expandindo os produtos, que $C_{12} = M_3 + M_5$ e $C_{21} = M_2 + M_4$ valem para blocos quaisquer. Por que é essencial que nenhuma das contas use a comutatividade da multiplicação?

(c) Escreva as recorrências da divisão em blocos (8 produtos) e de Strassen e resolva-as. Quantas multiplicações escalares cada uma faz para $n = 4$ e para $n = 2^k$?

(d) Suponha que alguém descubra como multiplicar matrizes 3×3 com k multiplicações (sem usar comutatividade). Qual é o maior k que tornaria o algoritmo recursivo resultante assintoticamente mais rápido que Strassen? A construção de Laderman (1976), com $k = 23$, consegue?

(e) Como aplicar Strassen quando n não é potência de 2? E por que, mesmo quando n é potência de 2, as implementações trocam para o algoritmo $\Theta(n^3)$ abaixo de certo tamanho?

Parte II — Programação dinâmica: as ideias e os primeiros problemas

Exercício 5. Subproblemas repetidos e subestrutura ótima.

(a) A função recursiva ingênua `fib(n)`, que devolve `n` se `n <= 1` e `fib(n-1) + fib(n-2)` caso contrário, faz $\mathrm{ch}(n)$ chamadas para calcular F_n. Escreva a recorrência de $\mathrm{ch}(n)$ e prove por indução que $\mathrm{ch}(n) = 2F_{n+1} - 1$. Quantas chamadas são feitas para $n = 5$ e para $n = 30$ (use $F_{31} = 1\,346\,269$)?

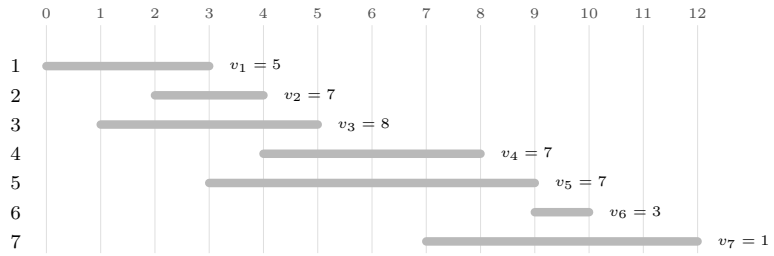
(b) Quantas chamadas faz a versão com *memoization* para o mesmo n? E quanto espaço extra precisa a versão *bottom-up* se guardar só o necessário?

(c) Diga, justificando em uma ou duas frases, quais problemas abaixo têm subestrutura ótima: *(i)* caminho mínimo entre dois vértices com pesos não negativos; *(ii)* caminho *simples* mais longo entre dois vértices; *(iii)* mochila 0/1; *(iv)* maior subsequência comum. Para o que não tiver, exiba um contraexemplo pequeno.

(d) Enuncie os quatro passos para desenvolver um algoritmo de programação dinâmica e aponte, em cada passo, o que foi feito na LCS.

Exercício 6. Seleção de intervalos ponderados.

Cada intervalo j tem início s_j, término f_j e valor v_j; dois intervalos são compatíveis quando não se sobrepõem (um pode começar no instante exato em que o outro termina). Os sete intervalos abaixo já estão em ordem de término, (s_j, f_j, v_j): $(0,3,5)$, $(2,4,7)$, $(1,5,8)$, $(4,8,7)$, $(3,9,7)$, $(9,10,3)$, $(7,12,1)$.



(a) Calcule $p(j)$ para $j = 1, \ldots, 7$.

(b) Preencha o vetor $M[0..7]$ com $\mathrm{OPT}(j) = \max\big(v_j + \mathrm{OPT}(p(j)),\ \mathrm{OPT}(j-1)\big)$, mostrando a conta de cada posição.

(c) Reconstrua o conjunto ótimo a partir de M, explicando cada decisão.

(d) Rode dois gulosos: *(i)* o que escolhe sempre o compatível que termina primeiro; *(ii)* o que escolhe sempre o compatível de maior valor. Quanto cada um obtém?

(e) Quanto custa o algoritmo completo, incluindo a ordenação e o cálculo de $p(\cdot)$? Por que a recursão *sem* memoization pode ser exponencial? Dê uma família de instâncias que provoque isso.

Exercício 7. Mochila 0/1.

Uma mochila tem capacidade $W = 7$, e há quatro itens com $(w_i, v_i) = (2,20)$, $(5,6)$, $(3,21)$, $(4,22)$, nessa ordem.

(a) Escreva a recorrência de $\mathrm{OPT}(i, c)$ e preencha a tabela $M[0..4][0..7]$.

(b) Reconstrua um conjunto ótimo a partir da tabela.

(c) O guloso que escolhe pela maior razão v_i/w_i acerta nesta instância? E na versão *fracionária* do problema, qual é o valor ótimo?

(d) O custo $\Theta(nW)$ é polinomial no tamanho da entrada? Explique o termo *pseudopolinomial*, considerando quantos bits são necessários para escrever W.

(e) Escreva em C a versão que usa um único vetor `m[0..W]`. Em que ordem a capacidade c deve ser percorrida? O que a sua função passa a calcular se a ordem for invertida? Qual seria o resultado para esta instância nesse caso?

Parte III — Programação dinâmica em sequências e em grafos

Exercício 8. Maior subsequência comum e distância de edição.

Considere $X =$ `PALETA` e $Y =$ `TAPETE`.

(a) Preencha a tabela $c[0..6][0..6]$ da maior subsequência comum e dê $c[6,6]$.

(b) A LCS é única? Encontre todas as subsequências comuns de comprimento máximo, mostrando na tabela os caminhos de reconstrução que levam a cada uma.

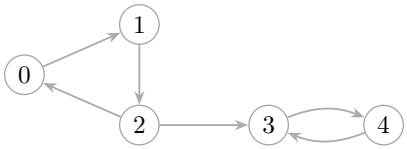
(c) Preencha a tabela $E[0..6][0..6]$ da distância de edição (inserção, remoção e substituição de custo 1) e exiba um alinhamento ótimo.

(d) Se só fossem permitidas inserções e remoções, a distância seria $m+n-2\,|\mathrm{LCS}(X,Y)|$. Justifique essa fórmula e calcule-a para `PALETA`/`TAPETE` e para `TRATOR`/`RETRATO` (cuja LCS é `TRATO`). Em qual dos dois pares a substituição faz diferença, e por quê?

(e) Escreva em C uma função que calcula $c[m,n]$ guardando apenas duas linhas da tabela. Por que essa economia impede a reconstrução direta da subsequência?

Exercício 9. Fecho transitivo (Warshall).

Considere o digrafo G_3 com os arcos `0-1`, `1-2`, `2-0`, `2-3`, `3-4`, `4-3`:



(a) Escreva a recorrência de Warshall para $r_{ij}^{(k+1)}$ e calcule as matrizes $R^{(0)}, R^{(1)}, \ldots, R^{(5)}$, destacando as entradas que mudam em cada rodada.

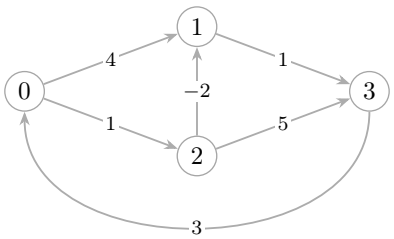
(b) Use o fecho para listar os componentes fortemente conexos de G_3. Que critério sobre $R^{(5)}$ você usou?

(c) Por que as entradas da diagonal $r_{ii}^{(5)}$ podem valer 1 mesmo sem laços no grafo? O que $r_{ii}^{(5)} = 1$ significa?

(d) O fecho também pode ser obtido com uma busca (em largura ou em profundidade) a partir de cada vértice. Compare o custo das duas abordagens em um grafo esparso ($A = \Theta(V)$) e em um denso ($A = \Theta(V^2)$).

Exercício 10. Caminhos mínimos entre todos os pares (Floyd).

Considere o digrafo com pesos G_4 abaixo, com os arcos `0-1` (4), `0-2` (1), `1-3` (1), `2-1` (-2), `2-3` (5) e `3-0` (3):



(a) G_4 tem um arco de peso negativo. Verifique que ele não tem ciclo de peso negativo, listando os seus ciclos.

(b) Calcule $D^{(0)}, D^{(1)}, \ldots, D^{(4)}$ pela recorrência $d_{ij}^{(k+1)} = \min\big(d_{ij}^{(k)},\ d_{ik}^{(k)} + d_{kj}^{(k)}\big)$, destacando as entradas que mudam em cada rodada.

(c) Mantenha também a matriz *prox* (primeiro vértice depois de i no melhor caminho até j) e use-a para reconstruir os caminhos mínimos de 0 a 3 e de 1 a 2.

(d) Troque o peso do arco `3-0` de 3 para -1. Que ciclo passa a ter peso negativo? Como o algoritmo revela isso, e o que acontece com as "distâncias" que ele calcula?

(e) Um colega escreveu os três laços na ordem `i`, `j`, `k`, com o laço de `k` por dentro. Mostre que essa versão erra no caminho `0-1`, `1-3`, `3-2` com pesos 1, indicando qual entrada sai errada e por quê.

Para estudar

Divisão e conquista e teorema mestre: Levitin (2012), cap. 5; Cormen et al. (2009), §4.3–4.5; Kleinberg e Tardos (2006), §5.1–5.2.
Par de pontos mais próximo: Kleinberg e Tardos (2006), §5.4; Cormen et al. (2009), §33.4; Shamos e Hoey (1975).
Karatsuba: Karatsuba e Ofman (1962); Levitin (2012), §5.4; Kleinberg e Tardos (2006), §5.5; Dasgupta et al. (2008), §2.1.
Strassen: Strassen (1969); Levitin (2012), §5.4; Cormen et al. (2009), §4.2.
Programação dinâmica, memoization e intervalos ponderados: Kleinberg e Tardos (2006), §6.1–6.2; Cormen et al. (2009), §15.3; Bellman (1957).
Mochila: Kleinberg e Tardos (2006), §6.4; Levitin (2012), §8.2; Dasgupta et al. (2008), §6.4.
LCS e distância de edição: Cormen et al. (2009), §15.4; Dasgupta et al. (2008), §6.3; Kleinberg e Tardos (2006), §6.6; Wagner e Fischer (1974).
Warshall e Floyd: Cormen et al. (2009), §25.2; Levitin (2012), §8.4; Dasgupta et al. (2008), §6.6; Warshall (1962); Floyd (1962).